An Ablation-Based Benchmark for Automated Trace Error Analysis

Aditya Vidhate

August 25, 2026

Abstract

Automated error-analysis agents ("Engine") promise to read production traces of AI applications and surface the failures hiding in them, but evaluating such an agent is hard: real traces come without ground-truth labels. We present an end-to-end benchmark that manufactures its own ground truth by *ablation*: known errors are injected into otherwise-clean traces of a real LangGraph application, and the Engine's predicted issueboard is scored against the injected record. The pipeline generates a stratified synthetic input corpus, collects traces through a black-box harness, injects errors under two mechanisms (post-hoc content corruption and dependency-fault re-execution), and scores predictions with an exact-key matcher across four independent axes: category detection, per-error localization, severity calibration, and description fidelity. On a 398-trace single-turn corpus with 14 injected errors (70 occurrences, 16.3% prevalence), a `gpt-5-mini` Engine achieves a macro category F1 of 0.09 and a matched-error F1 of 0.32; localization precision is 1.0 on every error it detects, but recall collapses on errors whose evidence is an *absence* rather than a visible contradiction, and three of seven categories are never correctly detected despite the full taxonomy being provided to the Engine in its prompt. The benchmark's disaggregated design attributes these failures to specific, actionable gaps.

1 Introduction

An Engine-style agent ingests execution traces of an AI application and returns an *issueboard*: a set of canonical issues, each with a category, severity, description, and the trace locations where it occurs. Benchmarking such a system faces a bootstrapping problem: knowing which errors are truly present in a trace corpus requires exactly the analysis the Engine is supposed to automate.

We resolve this with an ablation methodology. Starting from traces of a healthy application, we inject a set of *known errors* E_K with full bookkeeping of what was injected where. The Engine analyzes the corrupted corpus blind; its predictions are scored against the injection record. The corpus may also contain *hidden errors* E_h — genuine defects of the target application that we never injected — and the scoring design accounts for them explicitly (§2.5).

Our contributions are: (i) a fully app-agnostic benchmark pipeline — the target application is reached only through a configuration file and LangGraph's standard server surfaces; (ii) a dual-mechanism injection engine distinguishing *content* errors from *mechanism* errors; (iii) an exact-key scoring scheme that removes text-matching ambiguity from the primary metrics; and (iv) an evaluation of a tool-using `gpt-5-mini` Engine on a 398-trace corpus, with a failure analysis the disaggregated metrics make possible.

2 Method

2.1 Pipeline

The system runs five stages, each writing checkpointed artifacts:

1. **Input generation.** A dimension grid (topic, length, ambiguity, language, complexity, user goal) crossed with personas — including adversarial personas (prompt injection, jailbreak, scope creep) — is expanded by an LLM into concrete user requests, stratified so every cell is populated.

2. **Trace harness.** Each input is run against the target application through its LangGraph server. Traces are collected from the tracing backend behind a `TraceStore` boundary, normalized into a backend-agnostic schema, and persisted. Failed collections are quarantined per-input rather than aborting the batch.

3. **Ablation.** An LLM ablation agent proposes concrete errors under a fixed public taxonomy; each proposal is validated by a dry run and then injected into a disjoint sample of traces (§2.2).

4. **Engine under test.** The Engine receives only a leak-stripped trace export, the category taxonomy, and an (empty) seed issueboard. It returns a predicted issueboard.

5. **Scoring.** Predictions are matched to ground truth and scored on four axes (§2.4).

2.2 Ground truth by ablation

Errors are injected under two mechanisms:

Mode A — `replay_edit` (content). The trace's recorded content is corrupted directly — e.g., truncating an answer mid-sentence or splicing in an invented policy — with span-level consistency maintained so the corruption is not trivially detectable as a format artifact. On multi-turn traces this uses checkpoint time-travel to fork and re-execute downstream turns; on single-turn traces it is a consistency-managed post-hoc edit.

Mode C — `dependency_fault` (mechanism). The input is re-run against the application with a declared dependency shim armed (e.g., the retriever returns stale documents, the ticket tool fails internally). Activation is verified by diffing against the baseline trace; the resulting error is organic — the model's own reaction to the fault.

Every injection is recorded as an `IssueOccurrence` keyed by `(trace_id, category_id)`. Two errors of the same category are never injected into the same trace (*same-category disjointness*), which is what makes that key unique and enables exact matching. Injections that the application self-corrects are detected and discarded rather than shipped as false ground truth.

Prevalence is controlled by a stratified input-level split: 30% of inputs are *control* — never ablated, carried through byte-identical — so that precision and recall are interpretable against a known base rate and false-positive behavior on clean traces is measurable.

2.3 Leak prevention

The Engine must not be able to detect injections from side channels. The export path allowlist-copies fields, strips ablation bookkeeping, scans for injection-vocabulary tokens, and normalizes timestamps to a per-trace epoch so that edited spans carry no temporal fingerprint. Benchmark-side LLM calls run with tracing disabled so the Engine's tracing project contains only the target application's genuine activity.

2.4 Scoring

A predicted occurrence is matched to ground truth by the exact key `(trace_id, category_id)`; a TF-IDF text fallback exists only for wrong-category predictions and its usage rate is reported as a reliability statistic. Matched occurrences are grouped into issue-level pairs by argmax overlap. Four scorers report independently — a single composite would hide which axis failed:

1. **Category detection**: per-category precision/recall/F1 and Cohen's κ over trace-level presence (κ matters because prevalence is low — 5 injected traces per error in a 398-trace corpus).
2. **Per-error localization**: the same metrics per injected error.
3. **Severity calibration**: an asymmetric loss — under-calling severity is penalized quadratically, over-calling linearly.
4. **Description fidelity**: TF-IDF cosine between predicted and ground-truth descriptions, in $[0, 1]$.

2.5 Bias: precision is a lower bound

Because the corpus may contain genuine defects we did not inject (E_h), a prediction matching no injection is not necessarily wrong. All such predictions are counted *against* precision and simultaneously surfaced as an auditable "E_h candidate" list. Reported precision is therefore a lower bound on true precision; recall over E_K is unbiased, since every injected error is known.

3 Experimental setup

Target application. A customer-support agent (LangGraph `create_react_agent`) with a RAG retrieval tool over a product knowledge base and a ticket-creation tool, served via `langgraph` server and traced to a dedicated LangSmith project.

Corpus. 400 single-turn inputs from a 420-cell generation grid (safe and adversarial personas); 398 traces collected, 2 quarantined by the harness. Split: 120 control / 280 ablate inputs across 11 strata (seed 20260824).

Ground truth. The ablation agent proposed 2 errors per category across the 7-category taxonomy; all 14 validated and injected at 5 occurrences each (70 occurrences; 65 distinct injected traces; 16.3% trace-level prevalence; 7 errors per mode).

Engine. A LangGraph agent on `gpt-5-mini` that inspects each trace through four read-only tools (`get_trace`, `list_spans`, `read_span`, `search_text`), analyzes traces in parallel batches of 16, then consolidates per-trace findings into canonical issues in a clustering-and-merge step.

Its system prompt contains the full category vocabulary (names and descriptions) and running issue titles for cross-trace consistency — the concrete injected errors are never revealed.

Runtime. Harness 2.6 h, ablation 0.7 h, Engine 2.1 h, scoring <1 s; 5.4 h total on one machine.

4 Results

4.1 Headline

Metric	Value
Category F1 (macro)	0.094
Category precision (macro)	0.075
Category recall (macro)	0.257
Matched-error F1 (macro)	0.315
Mean severity loss (asymmetric)	0.100
Mean description score	0.190

Table 1: Headline metrics, reported separately by design. The gap between category-level and matched-error F1 already carries information: when the Engine finds an injected error it localizes it well, but most predictions land on uninjected traces.

Matcher reliability. 442 of 559 predicted occurrences resolved by exact key; 117 (20.9%) needed the text fallback; 540 resolved to no injection and enter the E_h/false-positive pool. The primary numbers are thus dominated by exact bookkeeping, not text similarity.

4.2 Category detection

Category	Precision	Recall	F1	κ	Support
formatting	0.364	0.400	0.381	0.364	10
hallucination	0.082	0.500	0.141	0.102	10
instruction_violation	0	0	0	−0.048	10
other	0	0	0	−0.017	10
retrieval_failure	0.058	0.400	0.101	0.060	10
state_loss	0	0	0	0.000	10
tool_misuse	0.018	0.500	0.035	−0.014	10

Table 2: Scorer 1 — trace-level category detection. Support = injected traces per category (2 errors × 5).

4.3 Per-error localization

Table 3 breaks detection down to the 14 injected errors. The pattern that the category table blurs is stark here: every category with a nonzero row owes it to exactly one of its two errors being found reliably, while its sibling sits at recall 0.2 or 0.

Error	Mode	Prec.	Recall	F1	κ
E-formatting-00 (mid-sentence truncation)	C	1	0.8	0.889	0.888
E-formatting-01 (garbled bullet markers)	A	0	0	0	0
E-hallucination-00 (invented refund policy)	A	1	0.8	0.889	0.888
E-hallucination-01 (answer from unrelated docs)	C	1	0.2	0.333	0.331
E-instruction_violation-00 (quiet adversarial compliance)	A	0	0	0	0
E-instruction_violation-01 (dropped safety disclaimer)	C	0	0	0	0
E-other-00 (ticket success despite internal failure)	C	0	0	0	0
E-other-01 (fabricated escalation path)	A	0	0	0	0
E-retrieval_failure-00 (ignores retrieved policy)	A	1	0.6	0.750	0.748
E-retrieval_failure-01 (stale retriever corpus)	C	1	0.2	0.333	0.331
E-state_loss-00 (forgotten ticket fields)	A	0	0	0	0
E-state_loss-01 (contradictory guidance)	C	0	0	0	0
E-tool_misuse-00 (ticket missing required details)	C	1	0.8	0.889	0.888
E-tool_misuse-01 (fabricated ticket, no tool call)	A	1	0.2	0.333	0.331

Table 3: Scorer 2 — per-error localization (support 5 each). Mode A = content corruption (`replay_edit`); Mode C = mechanism fault (`dependency_fault`). Precision is 1.0 on *every* detected error: the exact-key design means a correct-category flag on an injected trace is never ambiguous.

4.4 Severity and descriptions

Over the 5 matched issue pairs, mean asymmetric severity loss is 0.100; the single disagreement over-called a `medium` as `high` (the cheap direction under the asymmetric loss — no injected error's severity was under-called). Description similarity averages 0.19 across the four well-detected errors (range 0.155–0.223): the Engine describes the same defects in its own operational vocabulary rather than the ground truth's, which the TF-IDF backend scores conservatively (§6).

5 Analysis

The taxonomy was known; the zero rows are genuine gaps. We verified in the shipped code path that the Engine receives the full 7-category vocabulary — exact ids, names, and descriptions — in the system prompt of its per-trace analysis step and again in both consolidation steps. The three all-zero categories in Table 2 are therefore not vocabulary mismatches but detection failures, and they fail in three distinct ways:

- `state_loss` was never predicted *anywhere* in the run — zero issues, zero occurrences, κ exactly 0. The category description defines it as a failure "across turns," and the corpus is single-turn; the injected manifestations (forgotten ticket fields, contradictory guidance within one answer) are legitimate state failures, but the Engine appears to treat the category as structurally inapplicable to single-turn traces. This is as much a taxonomy–corpus interaction as an Engine deficiency, and the benchmark surfaces it precisely.
- `instruction_violation` shows the opposite failure: the Engine predicted it *often* (6 of its 20 issues) but never on an injected trace — yielding zero precision *and* recall with negative κ (worse than chance). Its predictions were dominated by one 78-occurrence issue flagging the application's own system prompt appearing in recorded model-call inputs — a property of the trace format, explicitly excluded by its instructions — rather than the injected quiet compliance with an adversarial override.

- `other`, the escape hatch, was used (3 predicted issues) but never for the two injected `other` errors, both of which require correlating a tool span's internal failure with the answer's claim of success.

Salience, not injection mechanism, predicts detection. Detected and missed errors split almost evenly across Modes A and C (detected: 3A/4C; missed: 4A/3C), so the Engine is not simply better at content or mechanism errors. The discriminating feature is whether evidence is a *visible contradiction* or an *absence*. Every well-detected error leaves a contradiction on the surface of the trace: an answer cut mid-sentence, an invented policy contradicting a retrieved document, a ticket confirmation with no ticket-tool call. Every fully missed error requires noticing what is *not* there: a disclaimer that should have been present, fields the user requested that the ticket silently dropped, a refusal that should have happened. This suggests a concrete Engine improvement — per-trace expectation checklists derived from the system prompt — that the benchmark can now measure.

Recall-0.2 errors mark a second, softer boundary. Three errors were found exactly once out of five (`hallucination-01`, `retrieval_failure-01`, `tool_misuse-01`). All three are detectable only by cross-referencing the answer against retrieval or tool spans; the Engine's per-trace tool budget appears to bound how often it performs that check.

The E_h pool is doing its job. Of 540 unmatched occurrences, most belong to the instrumentation-leak issue above (a scoring-time false positive, but an actionable finding about the trace export). A minority are plausibly real hidden defects of the target application — e.g., an arithmetic error in a proration calculation and a user-supplied webhook never forwarded to the ticket tool — exactly the discoveries the biased-precision design chooses to surface for audit rather than silently penalize.

6 Limitations and future work

The corpus is single-turn, which the `state_loss` result shows is a real constraint; multi-turn ablation is implemented (checkpoint time-travel) but requires the collection server's lifetime to span the run — a persistent checkpointer is the planned fix. Reported precision is a lower bound (§2.5); tightening it requires auditing the E_h pool. The description scorer's TF-IDF backend under-credits semantically-correct paraphrase; an LLM-judge backend exists behind the same interface. Finally, the benchmark is built for model comparison — both Engine arms share every byte of prompt and differ only in model id — and a `gpt-5.1` arm over the same frozen traces and ground truth is the immediate next experiment.

7 Conclusion

Ablation converts the unlabeled-trace problem into a controlled experiment: known errors in, predictions out, exact-key bookkeeping in between. On this corpus the resulting picture of a `gpt-5-mini` Engine is sharp and actionable: perfect localization precision on the errors it sees, systematic blindness to failures of omission, one category never attempted, one category drowned by a single false-positive mode, and a hidden-error audit trail. None of this is visible in a single composite score — which is the argument for the benchmark's disaggregated design.